VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

RAJINDER KUMAR (1BM23CS260)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sept-2025 to Jan-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled " Bio Inspired Systems (23CS5BSBIS)" carried out by **RAJINDER KUMAR (1BM23CS260),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Mayanka Gupta Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

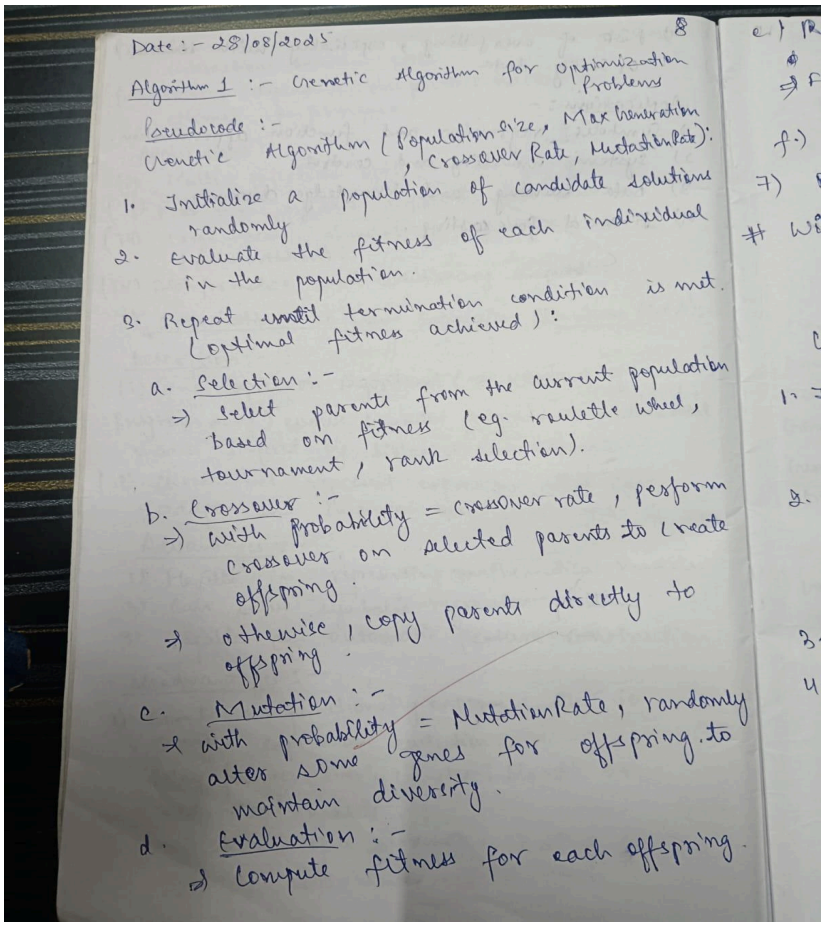
Github Link:

https://github.com/rajinderkumar143/BIS-LAB.git

Program 1

Genetic Algorithm for Optimization Problems

Algorithm:



Date :- 28/08/2025 8

Algorithm 1 :- Genetic Algorithm for Optimization Problems

Pseudocode :-

Genetic Algorithm (PopulationSize, MaxGeneration, CrossoverRate, MutationRate):

1. Initialize a population of candidate solutions randomly
2. Evaluate the fitness of each individual in the population.
3. Repeat until termination condition is met. (Optimal fitness achieved):
   a. Selection :-
      => Select parents from the current population based on fitness (eg. roulette wheel, tournament, rank selection).
   b. Crossover :-
      => With probability = crossover rate, perform crossover on selected parents to create offspring.
      => otherwise, copy parents directly to offspring.
   c. Mutation :-
      => with probability = MutationRate, randomly alter some genes for offspring to maintain diversity.
   d. Evaluation :-
      => Compute fitness for each offspring.

e.) Replacement (Survivor Selection):-
* Compute fitness ~~for each offspring~~.
⇒ Form a new population by replacing old individuals with new offspring.

f.) Update Best Solutions.

7) Return the Best Solution found.

With application, Using the function maximization problem $f(x) = x^2$ with $x \in [0, 31]$ as example.

Genetic Algorithm (Popsize, Maxgen, CrossOverRate, mutationRate):

1. Initialize population []
   for i = 1 to Popsize:
   population[i] = random 5-bit binary string.
2. Evaluate fitness []
   for i = 1 to Popsize:
   x = decode (population[i])
   fitness[i] = f(x) = x^2.
3. best = best individual in population
4. for generation = 1 to Maxgen:
   # --Selection--
   parents[] = []
   for i = 1 to Popsize:
   parent = select based on fitness (population, fitness)
   parents.append(parent)

```
# -- Crossover --
offspring[] = []
for i = 1 to PopSize step 2:
    p1 = parents[i]
    p2 = parents[i+1]
    r = random(0,1)
    if r < crossoverRate:
        (child1, child2) = single_point_crossover
                           (p1, p2)
    else:
        child1 = p1
        child2 = p2
    offspring.append(child1)
    offspring.append(child2)
# -- Mutation --
for i = 1 to PopSize:
    for each bit in offspring[i]:
        r = random(0,1)
        if r < MutationRate:
            flip(bit)
# -- Evaluation --
for i = 1 to PopSize:
    x = decode(offspring[i])
    fitness[i] = f(x) = x^2
# -- Replacement --
population = offspring
# --- Update Best --
best_current = best in offspring
```

if (fitness (best current) > fitness (best)):
    best = best_current
5. Return best (solution and fitness).

Output

POP_SIZE = 20
CHROM_Length = 8
Max_Gen = 50
Crossover_Rate = 0.7
Mutation_Rate = 0.05

Generation 1: Best x = 247 | Fitness = 61018.94
Generation 2: Best x = 247 | Fitness = 61018.94
Generation 3: Best x = 251 | Fitness = 62997.78
Generation 4: Best x = 255 | fitness = 65019.94
Generation 5: Best x = 255 | fitness = 65019.94
Generation 6: Best x = 255 | fitness = 65019.94
:
Generation 50: Best x = 255 | fitness = 65019.94

Final Best Solution:
Chromosome: 11111111, x = 255, f(x) = 65019.94

Code:

```
import random
import math

POP_SIZE = 20
CHROM_LENGTH = 8
MAX_GEN = 50
CROSSOVER_RATE = 0.7
MUTATION_RATE = 0.05

def decode(chromosome):
    return int(chromosome, 2)

def fitness(chromosome):
    x = decode(chromosome)
    return x * x + 10 * math.sin(x)

def select(population):
    # Tournament selection
    tournament = random.sample(population, 3)
    return max(tournament, key=fitness)

def crossover(p1, p2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, CHROM_LENGTH - 1)
        return p1[:point] + p2[point:], p2[:point] + p1[point:]
    return p1, p2

def mutate(chromosome):
    return ''.join(
        bit if random.random() > MUTATION_RATE else '1' if bit == '0' else '0'
        for bit in chromosome
    )

def genetic_algorithm():
    population = [''.join(random.choice("01") for _ in range(CHROM_LENGTH))
                  for _ in range(POP_SIZE)]
    best = max(population, key=fitness)

    for gen in range(MAX_GEN):
        parents = [select(population) for _ in range(POP_SIZE)]
        offspring = []
        for i in range(0, POP_SIZE, 2):
            c1, c2 = crossover(parents[i], parents[i+1])
            offspring.extend([mutate(c1), mutate(c2)])

        # Elitism
```

```
        worst_index = min(range(len(offspring)), key=lambda i: fitness(offspring[i]))
        offspring[worst_index] = best

        population = offspring
        current_best = max(population, key=fitness)
        if fitness(current_best) > fitness(best):
            best = current_best

        print(f"Generation {gen+1}: Best x = {decode(best)} | Fitness = {fitness(best):.2f}")

    return best

best_solution = genetic_algorithm()
print("\nFinal Best Solution:")
print(f"Chromosome: {best_solution}, x = {decode(best_solution)}, f(x) =
{fitness(best_solution):.2f}")
```

Program 2

Optimization via Gene Expression Algorithms

Algorithm:

Date :- 4/09/2025 12

Optimization Via Gene Expression :-

Algorithm :- Input :- objective fn (f(x)) to minimize or maximize.
Parameters :- Population size, No. of gens, chromosome length (L), Pm, Pc, selection method

Step 1 :- Initialize population → Generate random chromosomes (linear strings).

Step 2 :- Decode → Convert each chromosome into an expression tree (ET).

Step 3 :- Evaluate fitness → Calculate objective function value for each solution.

Step 4 :- Select parents → Choose better solutions using selection methods.

Step 5 :- Apply operators → Perform crossover, mutation and transposition to create new offspring.

Step 6 :- Form new population → Replace old population & ~~continue~~

Step 7 :- Repeat steps 2-6 until stopping condition (max generations is found or good fitness).

Step 8 :- Output best solution → chromosome with highest fitness.

Pseudocode :-

Input :- Objective function $f(x)$, population size P, chromosome length (L), max Generations, crossover Rate pc, mutation rate pm

Output :- Best solution found.

Begin

Initialize population with random chromosomes

For generation = 1 to G do :

For each chromosome in Population do:

Decode chromosome into expression tree

Evaluate fitness using $f(x)$

End For

Select parents based on fitness

Apply crossover to parents with Probability. pc → create offspring.

Apply mutation to offspring with Probability pm

Apply transposition/ insertion if required

Form new population from offspring.

End For

Return Best chromosome with highest fitness

End.

MG
4/9/25

Output

Travelling Salesman Problem :-
(Genetic Algorithm)

Enter number of cities (eg. 10-50): 20
Enter population size (eg. 50-200): 80
Enter number of generations (eg. 100-1000): 200

Running Genetic Algorithm...

Best Tour Found:
Tour (City Indices): [12, 14, 8, 19, 12, 1, 13, 16,
11, 0, 10, 15, 5, 4, 18, 3, 17,
7, 8, 9]

Total Distance: 458.76.

MG.

Code:

```
import random
import numpy as np

# --------------------------
# Function Definitions
# --------------------------

def create_cities(num_cities):
    return [(random.uniform(0, 100), random.uniform(0, 100)) for _ in range(num_cities)]

def distance(city1, city2):
    return np.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)

def total_distance(tour, cities):
    return sum(distance(cities[tour[i]], cities[tour[(i+1) % len(tour)]]) for i in range(len(tour)))

def generate_population(pop_size, num_cities):
    return [random.sample(range(num_cities), num_cities) for _ in range(pop_size)]

def fitness(tour, cities):
    return 1 / total_distance(tour, cities)

def tournament_selection(population, cities, k=3):
    selected = random.sample(population, k)
    selected.sort(key=lambda t: fitness(t, cities), reverse=True)
    return selected[0]

def crossover(parent1, parent2):
    size = len(parent1)
    a, b = sorted(random.sample(range(size), 2))
    child = [-1] * size
    child[a:b] = parent1[a:b]
    fill = [item for item in parent2 if item not in child]
    ptr = 0
    for i in range(size):
        if child[i] == -1:
            child[i] = fill[ptr]
            ptr += 1
    return child

def mutate(tour, mutation_rate):
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(tour)), 2)
        tour[i], tour[j] = tour[j], tour[i]
    return tour
```

```
# ---------------------------
# Genetic Algorithm Function
# ---------------------------

def genetic_algorithm_tsp(num_cities, pop_size, generations, crossover_rate=0.9,
mutation_rate=0.02):
    cities = create_cities(num_cities)
    population = generate_population(pop_size, num_cities)
    best_tour = min(population, key=lambda t: total_distance(t, cities))

    for gen in range(generations):
        new_population = []
        for _ in range(pop_size):
            parent1 = tournament_selection(population, cities)
            parent2 = tournament_selection(population, cities)
            if random.random() < crossover_rate:
                child = crossover(parent1, parent2)
            else:
                child = parent1[:]
            child = mutate(child, mutation_rate)
            new_population.append(child)

        population = new_population
        current_best = min(population, key=lambda t: total_distance(t, cities))
        if total_distance(current_best, cities) < total_distance(best_tour, cities):
            best_tour = current_best

    return best_tour, total_distance(best_tour, cities), cities

# ---------------------------
# Main Program with Input/Output
# ---------------------------

def main():
    print("Travelling Salesman Problem - Genetic Algorithm")

    try:
        num_cities = int(input("Enter number of cities (e.g., 10 - 50): "))
        pop_size = int(input("Enter population size (e.g., 50 - 200): "))
        generations = int(input("Enter number of generations (e.g., 100 - 1000): "))
    except ValueError:
        print("Please enter valid integers.")
        return

    print("\nRunning Genetic Algorithm...")
    best_tour, best_distance, cities = genetic_algorithm_tsp(num_cities, pop_size, generations)

    print("\n Best Tour Found:")
```

```
    print("Tour (city indices):", best_tour)
    print(f"Total Distance: {best_distance:.2f}")

    # Optional: Plot the tour
    try:
        import matplotlib.pyplot as plt
        x = [cities[i][0] for i in best_tour] + [cities[best_tour[0]][0]]
        y = [cities[i][1] for i in best_tour] + [cities[best_tour[0]][1]]
        plt.figure(figsize=(8, 6))
        plt.plot(x, y, 'o-', color='blue')
        plt.title(f"TSP Best Tour (Distance: {best_distance:.2f})")
        plt.xlabel("X")
        plt.ylabel("Y")
        plt.grid(True)
        plt.show()
    except ImportError:
        print("\nTo visualize the tour, install matplotlib:\n  pip install matplotlib")

# ---------------------------
# Run the Program
# ---------------------------

if __name__ == "__main__":
    main()
```

Program 3

Particle Swarm Optimization for Function Optimization

Algorithm:

Date:- 11/09/2025 15

Particle Swarm Optimization (PSO) for function Optimization :-

Algorithm :-

1) "Initialize" the swarm with random positions and velocities.
2) "Evaluate" each particles' s fitness using the objective function.
3) "Update personal and global bests.
4) "Update" velocities using the PSO equation:-

$$v(t+1) = w * v(t) + c1 * r1 * (pbest - x(t)) + c2 * r2 * (gbest) - x(t))$$

where :-

- w = inertia weight (controls exploration vs exploitation)
- c_1, c_2 = acceleration coefficients (cognitive and social parameters).
- r_1, r_2 = random numbers between 0 and 1.
- pbest = particle's personal best position
- gbest = global best position

Pseudocode :-

Input : objective_function, swarm_size, max_iterations, w, c1, c2, bounds

BEGIN

FOR i = 1 To swarm_size DO

position(i) = random(bounds),

```
velocity(i) = random_small(i).
gbest(i) = position(i);
END FOR
gbest = best_particle_position()

FOR t = 1 to max_iterations DO
  FOR i = 1 to swarm_size DO
    r1, r2 = random(0,1)
velocity(i) = w*velocity(i) + c1*r1*(pbest(i)
    -position(i)) + c2*r2*(gbest-
    position(i))
position(i) = position(i) + velocity(i)
If fitness(position(i)) < fitness(pbest(i))
    THEN
pbest(i) = position(i)
If fitness(position(i) < fitness(gbest)
    THEN
      gbest = position(i)
      END FOR
  END FOR
  RETURN gbest
END.
```

DAA

Advantages :- 1) Simple implementation
2) Relatively few parameters to tuned.
3) Higher efficiency and Probability to find the global optima.
4) No overlapping or mutation.
5) Low computational time.

Disadvantages :
1) Difficulty to initialize control parameters.
2) Premature convergence & trapping into the local minima especially on solving high dimensional problems.

Output

Application :- Economic Load dispatch (ELD) problem using particle swarm optimization.

The ELD problem aims to minimize the total cost of generation while satisfying the power demand and operational constraints of each generator.

```
generator_data = np.array([[0.008, 7.0, 200.0, 10.0, 85.0],
                           [0.009, 6.3, 180.0, 10.0, 80.0],
                           [0.007, 6.8, 140.0, 10.0, 70.0]])
```

```
Iteration 50: Best fitness = 1580.4029
Iteration 100: Best fitness = 1579.6848
Iteration 150: Best fitness = 1579.6848
Iteration 200: Best fitness = 1579.6848
```

--- Optimization Complete ---

Optimal Power Generation (MW):

Generator 1: 31.9359 MW

Generator 2: 67.8764 MW

Generator 3: 50.7839 MW

Total Power Generated: 149.9962 MW

Total Load Demand: 150.0 MW

Minimal Total Cost Generation: $1579.67

Code:

```
import numpy as np
import random

# --- 1. Problem Definition (ELD) ---
# Data for a 3-unit system (from a common test case in research papers)
# [a, b, c, P_min, P_max] for each generator
generator_data = np.array([
    [0.008, 7.0, 200.0, 10.0, 85.0],
    [0.009, 6.3, 180.0, 10.0, 80.0],
    [0.007, 6.8, 140.0, 10.0, 70.0]
])

# Total power demand
total_demand = 150.0 # in MW

# The objective function to minimize (total generation cost)
def objective_function(P_values):
    """
    Calculates the total fuel cost for a given power output for each generator.
    P_values is a NumPy array of power outputs.
    """
    a = generator_data[:, 0]
    b = generator_data[:, 1]
    c = generator_data[:, 2]

    total_cost = np.sum(a * P_values**2 + b * P_values + c)
    return total_cost

# Penalty function for constraints
def calculate_fitness(P_values, total_demand, penalty_factor=1000):
    """
    Calculates the fitness of a particle, including penalties for violating constraints.
    - P_values: the current power output of generators
    - total_demand: the system load
    - penalty_factor: a large number to penalize constraint violations
    """
    # Sum of power generated
    generated_power = np.sum(P_values)

    # Calculate objective value (total cost)
    cost = objective_function(P_values)

    # Add a penalty for not meeting the power balance constraint
    # We use the squared difference to make the penalty more significant
    penalty = penalty_factor * (generated_power - total_demand)**2
```

```
    return cost + penalty

# --- 2. PSO Algorithm Implementation ---
class Particle:
    def __init__(self, num_dims, bounds):
        self.position = np.random.uniform(bounds[:, 0], bounds[:, 1], num_dims)
        self.velocity = np.zeros(num_dims)
        self.personal_best_position = np.copy(self.position)
        self.personal_best_fitness = float('inf')

    def update_velocity(self, global_best_position, w=0.5, c1=2.0, c2=2.0):
        r1 = random.uniform(0, 1)
        r2 = random.uniform(0, 1)

        # Cognitive component: towards its own best position
        cognitive_velocity = c1 * r1 * (self.personal_best_position - self.position)

        # Social component: towards the swarm's best position
        social_velocity = c2 * r2 * (global_best_position - self.position)

        self.velocity = w * self.velocity + cognitive_velocity + social_velocity

    def update_position(self, bounds):
        self.position += self.velocity

        # Clamp positions to stay within bounds
        self.position = np.clip(self.position, bounds[:, 0], bounds[:, 1])

def pso_optimizer(num_particles, max_iterations):
    num_generators = generator_data.shape[0]
    bounds = generator_data[:, 3:5]

    # Initialize the swarm
    particles = [Particle(num_generators, bounds) for _ in range(num_particles)]

    global_best_position = None
    global_best_fitness = float('inf')

    for i in range(max_iterations):
        for particle in particles:
            # Calculate fitness for the current position
            current_fitness = calculate_fitness(particle.position, total_demand)

            # Update personal best
            if current_fitness < particle.personal_best_fitness:
                particle.personal_best_fitness = current_fitness
                particle.personal_best_position = np.copy(particle.position)
```

```
            # Update global best
            if current_fitness < global_best_fitness:
                global_best_fitness = current_fitness
                global_best_position = np.copy(particle.position)

        # Update velocities and positions for all particles
        for particle in particles:
            particle.update_velocity(global_best_position)
            particle.update_position(bounds)

        # Print progress
        if (i+1) % 50 == 0:
            print(f"Iteration {i+1}: Best Fitness = {global_best_fitness:.4f}")

    return global_best_position, global_best_fitness

# --- 3. Run the Optimization ---
if __name__ == '__main__':
    num_particles = 30
    max_iterations = 200

    optimal_power, min_cost = pso_optimizer(num_particles, max_iterations)

    print("\n--- Optimization Complete ---")
    print("Optimal Power Generation (MW):")
    for i, p_gen in enumerate(optimal_power):
        print(f"  Generator {i+1}: {p_gen:.4f} MW")

    print(f"\nTotal Power Generated: {np.sum(optimal_power):.4f} MW")
    print(f"Total Load Demand: {total_demand} MW")
    print(f"Minimum Total Generation Cost: ${objective_function(optimal_power):.2f}")
```

Program 4

Ant Colony Optimization for the Traveling Salesman Problem

Algorithm:

Date :- 09/10/2025

Ant Colony Optimization for Travelling Salesman Problem (TSP) :-

Parameters :-

- m = no. of ants
- n = no. of cities
- α = influence of pheromone
- β = influence of heuristic (visibility)
- ρ = pheromone evaporation rate.
- Q = pheromone deposit constant.
- MaxIterations = no. of iterations
- τ = Pheromone value on an edge.
- η = Heuristic value (1/ distance)

Algorithm :-

1) Initialize
   - ⇒ Set parameters α, β, ρ, Q, number of ants and iterations.
   - ⇒ Initialize pheromone trails on all edges.
   - ⇒ Compute visibility = 1/ distance b/w cities.

2) Construct Solutions :-
   - ⇒ Each ant starts from a random city.
   - ⇒ At each step, choose the next city based on :-

$$P_{ij} = \frac{(\tau_{ij})^{\alpha} (\eta_{ij})^{\beta}}{\sum (\tau_{il})^{\alpha} (\eta_{il})^{\beta}}$$

3) Evaluate :- Calculate the total tour length for each ant.

4) Update Pheromone :- Evaporate old pheromone.

$\tau_{ij} = (1-\rho)\,\tau_{ij}$

$\tau_{ij} = \tau_{ij} + Q/L_k$.

5) Find Best Solution :- Record the best (shortest tour) so far.

6) Repeat :- continue until max no. of iterations is reached.

7) output :- Return the best tour and its total distance.

Pseudocode :-

Initialize pheromone trails = $\tau[i][j]$

Compute visibility $\eta[i][j] = 1/\text{distance}[i][j]$

For $t = 1$ to MaxIterations:

For each ant k:

Place ant k on a random start city

while not all cities visited:

choose next city j using probability:

$P(j) = (\tau[i][j]^{\alpha} * \eta[i][j]^{\beta}) / \Sigma(\tau[i][j]^{\alpha} * \eta[i][j]^{\beta})$.

Move to city j

END while

Compute tour lengths L_k

END for

For each edge (i, j):

$\tau[i][j] = (1-\rho) * \tau[i][j] + \Sigma(Q/L_k)$

for all ants using (i, j).

END for

Update best tour found

End for

Return best tour and its length.

MG

Output

Example: distance matrix for 5 cities (symmetric)

$$\begin{bmatrix} 0 & 2 & 9 & 10 & 7 \\ 2 & 0 & 6 & 4 & 3 \\ 9 & 6 & 0 & 8 & 5 \\ 10 & 4 & 8 & 0 & 6 \\ 7 & 3 & 5 & 6 & 0 \end{bmatrix}$$

n_ants = 10, n_iterations = 100,

alpha = 1, beta = 5, rho = 0.5, Q = 100

Iteration 1: Best length = 26.00

Iteration 2: Best length = 26.00

Iteration 3: Best length = 26.00

Iteration 4: Best length = 26.00

Iteration 5: Best length = 26.00

Iteration 6: Best length = 26.00

Iteration 7: Best length = 26.00

Iteration 8: Best length = 26.00

:

Iteration 100: Best length = 26.00

Best Tour: [3, 1, 0, 4, 2]

Best Tour length: 26.

MG
9/10/25

Code:

```
import numpy as np
import random

class AntColony:
    def __init__(self, distances, n_ants, n_iterations, alpha=1, beta=5, rho=0.5, Q=100):
        """
        distances: 2D numpy array of distances between cities
        n_ants: number of ants
        n_iterations: number of iterations to run
        alpha: pheromone importance
        beta: heuristic importance (usually distance)
        rho: pheromone evaporation rate
        Q: pheromone deposit factor
        """
        self.distances = distances
        self.pheromone = np.ones(self.distances.shape)  # initial pheromone levels
        self.all_inds = range(len(distances))
        self.n_ants = n_ants
        self.n_iterations = n_iterations
        self.alpha = alpha
        self.beta = beta
        self.rho = rho
        self.Q = Q

    def run(self):
        best_length = float('inf')
        best_tour = None

        for iteration in range(self.n_iterations):
            all_tours = self.construct_solutions()
            self.update_pheromones(all_tours)

            for tour, length in all_tours:
                if length < best_length:
                    best_length = length
                    best_tour = tour

            print(f"Iteration {iteration+1}: Best length = {best_length:.2f}")

        return best_tour, best_length

    def construct_solutions(self):
        all_tours = []
        for _ in range(self.n_ants):
            tour = self.construct_solution()
            length = self.tour_length(tour)
```

```
            all_tours.append((tour, length))
        return all_tours

    def construct_solution(self):
        tour = []
        visited = set()
        current_city = random.choice(self.all_inds)
        tour.append(current_city)
        visited.add(current_city)

        while len(visited) < len(self.distances):
            next_city = self.select_next_city(current_city, visited)
            tour.append(next_city)
            visited.add(next_city)
            current_city = next_city
        return tour

    def select_next_city(self, current_city, visited):
        pheromone = self.pheromone[current_city]
        distances = self.distances[current_city]

        probabilities = []
        for city in self.all_inds:
            if city not in visited:
                tau = pheromone[city] ** self.alpha
                eta = (1.0 / distances[city]) ** self.beta if distances[city] > 0 else 0
                probabilities.append(tau * eta)
            else:
                probabilities.append(0)
        probabilities = np.array(probabilities)
        probabilities_sum = np.sum(probabilities)

        if probabilities_sum == 0:
            # If all probabilities are zero, pick randomly from unvisited
            candidates = [city for city in self.all_inds if city not in visited]
            return random.choice(candidates)
        probabilities /= probabilities_sum
        return np.random.choice(self.all_inds, p=probabilities)

    def tour_length(self, tour):
        length = 0
        for i in range(len(tour)):
            from_city = tour[i]
            to_city = tour[(i + 1) % len(tour)]
            length += self.distances[from_city][to_city]
        return length

    def update_pheromones(self, all_tours):
```

```
        # Evaporate pheromone
        self.pheromone *= (1 - self.rho)

        for tour, length in all_tours:
            deposit = self.Q / length
            for i in range(len(tour)):
                from_city = tour[i]
                to_city = tour[(i + 1) % len(tour)]
                self.pheromone[from_city][to_city] += deposit
                self.pheromone[to_city][from_city] += deposit  # because symmetric TSP

# Example usage
if __name__ == "__main__":
    # Example distance matrix for 5 cities (symmetric)
    distances = np.array([
        [0, 2, 9, 10, 7],
        [2, 0, 6, 4, 3],
        [9, 6, 0, 8, 5],
        [10, 4, 8, 0, 6],
        [7, 3, 5, 6, 0]
    ])

    colony = AntColony(distances, n_ants=10, n_iterations=100, alpha=1, beta=5, rho=0.5, Q=100)
    best_tour, best_length = colony.run()

    print("Best tour:", best_tour)
    print("Best tour length:", best_length)
```

Program 5

Cuckoo Search (CS)

Algorithm:

Date :- 16/10/2025

11 Cuckoo Search :-

Algorithm :-

1) Initialize a population of host nests (candidate solutions).
2) Generate new solutions (cuckoo eggs) by performing Levy flights from existing solutions.
3) Evaluate fitness of new solutions.
4) Select a nest randomly and replace it with the new solution if the new one is better.
5) Abandon a fraction p_a, of the worst nests and create new ones randomly.
6) Keep the best solutions and rank them.
7) Repeat until maximum iterations or a stopping criterion is met.

Pseudo code :-

Algorithm Cuckoo Search ():
Input :-
n = population size of nests
p-a = fraction of nests to abandon
($0 < p\text{-}a < 1$).
Max Iterations = maximum allowed iterations.
f(x) = objective function to optimize.
Initialize 'n' host nests Xi ($i = 1, \ldots n$) with random solutions.
t = 0
while t < MaxIterations do
Generate a new solution by Levy flight:
cuckoo - i

Evaluate fitness $F_i = f(cuckoo\_i)$
Randomly choose a nest j among n
Evaluate fitness $F_j = f(X_j)$
if $F_i > F_j$ then
Replace X_j with cuckoo_j
end if
Abandon a fraction p_a of worse nests and build new ones randomly.
Keep the best solutions / nests.
Rank solutions and find the current best.

MG 16/10/25.

Cuckoo Search for Job Scheduling

Input :- Array = [2, 5, 1, 3, 4]
5 jobs with processing times:
- Job 0: 2 units
- Job 1: 5 units
- Job 2: 1 unit
- Job 3: 3 units
- Job 4: 4 units

Output :-

Iteration 100, Best total completion time : 35
Iteration 200, Best total completion time: 35
Iteration 300, Best total completion time: 35
:
Iteration 1000, Best total completion time: 35
Optimization finished,
Best job order : [2 0 3 4 1]
Best total completion time : 35.

MG 16/10/25.

Code:

```
import numpy as np
import math  # Import Python's math module

# Example processing times for n jobs
processing_times = np.array([2, 5, 1, 3, 4])  # Change this as needed
n_jobs = len(processing_times)

# Objective function: sum of completion times (total completion time)
def total_completion_time(order):
    completion_time = 0
    current_time = 0
    for job in order:
        current_time += processing_times[job]
        completion_time += current_time
    return completion_time

# Convert continuous vector to job permutation (random-key encoding)
def vector_to_permutation(vec):
    return np.argsort(vec)

# Levy flight step generator
def levy_flight(Lambda, dim):
    sigma = (math.gamma(1 + Lambda) * math.sin(math.pi * Lambda / 2) /
             (math.gamma((1 + Lambda) / 2) * Lambda * 2 ** ((Lambda - 1) / 2))) ** (1 / Lambda)
    u = np.random.normal(0, sigma, size=dim)
    v = np.random.normal(0, 1, size=dim)
    step = u / np.abs(v) ** (1 / Lambda)
    return step

# Parameters
n_nests = 25          # Number of nests (solutions)
pa = 0.25             # Probability of abandoning worse nests
n_iterations = 1000    # Number of iterations
dim = n_jobs          # Dimension of the problem (number of jobs)

# Initialize nests randomly in [0,1]
nests = np.random.rand(n_nests, dim)
fitness = np.array([total_completion_time(vector_to_permutation(nests[i])) for i in range(n_nests)])

# Find the best initial solution
best_idx = np.argmin(fitness)
best_nest = nests[best_idx].copy()
best_fitness = fitness[best_idx]

for t in range(n_iterations):
    # Generate new solutions via Levy flights
```

```
    for i in range(n_nests):
        step = levy_flight(1.5, dim)
        new_solution = nests[i] + 0.01 * step
        new_solution = np.clip(new_solution, 0, 1)  # Keep within [0,1]

        new_fitness = total_completion_time(vector_to_permutation(new_solution))

        # Replace if better
        if new_fitness < fitness[i]:
            nests[i] = new_solution
            fitness[i] = new_fitness

            if new_fitness < best_fitness:
                best_fitness = new_fitness
                best_nest = new_solution.copy()

    # Abandon some nests and generate new ones
    K = np.random.rand(n_nests, dim) > pa

    # Use a consistent permutation for difference calculation
    perm = np.random.permutation(n_nests)
    stepsize = np.random.rand(n_nests, dim) * (nests[perm] - nests[np.roll(perm, 1)])

    nests = nests + stepsize * K
    nests = np.clip(nests, 0, 1)

    # Evaluate new nests
    for i in range(n_nests):
        new_fitness = total_completion_time(vector_to_permutation(nests[i]))
        if new_fitness < fitness[i]:
            fitness[i] = new_fitness
            if new_fitness < best_fitness:
                best_fitness = new_fitness
                best_nest = nests[i].copy()

    # Print progress every 100 iterations
    if (t + 1) % 100 == 0:
        print(f"Iteration {t + 1}, Best total completion time: {best_fitness}")

print("\nOptimization finished.")
print("Best job order:", vector_to_permutation(best_nest))
print("Best total completion time:", best_fitness)
```

Program 6

Grey Wolf Optimizer (GWO)

Algorithm:

Date :- 23/10/2025

Grey Wolf Optimizer :-

- ⇒ metaheuristic algorithm optimization inspired by the social hierarchy and hunting behaviour of grey wolves.
- ⇒ Used for solving optimization problems - both continuous and discrete - similar to algorithms like PSO or GA.

Logic : Grey wolves live in packs and follows a strict "social hierarchy :-

Rank	Symbol	Role
Alpha	α	Leader (Best Sol)
Beta	β	Second leader (second Best sol)
Delta	δ	Third leader (3rd Best Sol)
omega	ω	followers (remaining wolves)

During hunting, the wolves encircle prey, attack and follow leaders (α, β, δ) to find the prey's position (ie the optimal sol^n).

Algorithm :-

Step 1:- Initialization

- ⇒ Randomly initialize the population of grey wolves (candidate solutions).
- ⇒ Each wolf represents a potential solution in the search space.

Step 2 : Fitness Evaluation

- ⇒ Evaluate the fitness (objective f^n value) of each wolf.
- ⇒ Identify the top three wolves: α (Best), β (second-best), and δ (third-best).

Step 3:- Position Updating (Hunting)

→ Other wolves (ω), update their positions based on α, β and δ using the following equations:-

$$\vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}_\alpha - \vec{X}|$$
$$\vec{D}_\beta = |\vec{C}_2 \cdot \vec{X}_\beta - \vec{X}|$$
$$\vec{D}_\delta = |\vec{C}_3 \cdot \vec{X}_\delta - \vec{X}|$$
$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha$$
$$\vec{X}_2 = \vec{X}_\beta - \vec{A}_2 \cdot \vec{D}_\beta$$
$$\vec{X}_3 = \vec{X}_\delta - \vec{A}_3 \cdot \vec{D}_\delta$$

X → represents the position of wolf in the search space

C → random coefficient

D → distance b/w current wolf's position & leader's wolf position

Finally, the wolf updates its position as:-

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3}$$

Step 4:- Coefficient vectors

These control the exploration and exploitation balance.

$$\vec{A} = 2 \cdot a \cdot \vec{r}_1 - a$$
$$\vec{C} = 2 \cdot \vec{r}_2$$

A → coefficient that controls the step size of the wolf's movement towards the prey (optimal sol).

where:-

(i) a decreases linearly from 2 to 0 over iterations.

(ii) $\vec{r}_1, \vec{r}_2 \in [0,1]$ are random vectors

Step 5:- Termination

(i) Repeat steps 2-4 until the stopping condition (eg. max^m iterations) is reached.

(ii) The α wolf represents the best-found solution.

Pseudocode :-

Initialize the population of grey wolves X_i ($i = 1, 2, \ldots N$).

Initialize a, A and C.

Evaluate the fitness of each search agent.

Identify X_α (best), X_β (second best) and X_δ (third best).

while (t < Max Iter)

For each wolf (X_i)

Update coefficient $a = 2 - t * (2 / \text{Max Iter})$

Generate random numbers $r_1, r_2 \in [0, 1]$

Compute $A = 2 * a * r_1 - a$

Compute $C = 2 * r_2$

Compute $D_\alpha = |C_1 * X_\alpha - X_i|$

Compute $D_\beta = |C_2 * X_\beta - X_i|$

Compute $D_\delta = |C_3 * X_\delta - X_i|$

Compute $X_1 = X_\alpha - A_1 \cdot D_\alpha$

Compute $X_2 = X_\beta - A_2 \cdot D_\beta$

Compute $X_3 = X_\delta - A_3 \cdot D_\delta$

Update position $X_i(t+1) = (X_1 + X_2 + X_3)/3$

End For

Evaluate fitness of all wolves.

Update $X_\alpha, X_\beta, X_\delta$

$t = t + 1$

End while

Return X_α as the best solution.

Application :- Feature Selection
↳ means choosing only the most important features that gives the best performance for a model

Input Data :- Iris dataset (Builtin with scikit-learn)
Iris dataset contains :-
(i) 150 samples (rows) of Iris flowers.
(ii) 4 features (columns) :-
1. Sepal length (cm) — 0
2. Sepal width (cm) — 1
3. Petal length (cm) — 2
4. Petal width (cm) — 3

wolf 1 : [0.8, 0.3, 0.9, 0.4]
wolf 2 : [0.2, 0.5, 0.3, 0.1]
Value > 0.5 → feature included
Value ≤ 0.5 → feature excluded

(iii) Target (output) :-
⇒ 0 : Iris-setosa
⇒ 1 : Iris-versicolor
⇒ 2 : Iris-virginica.

Output Total iterations = 20
Iteration 1/20, Best Accuracy : 1.0000
Iteration 2/20, Best Accuracy : 1.0000
⋮
Iteration 20/20, Best Accuracy : 1.0000
Best selected features : [0, 1, 2, 3]
Best Accuracy : 1.0

MG 23/10/25.

$$\text{Fitness} = \alpha \times (1 - \text{Accuracy}) + \beta \times \left(\frac{\text{No. of selected features}}{\text{Total features}}\right)$$

∴ Lower fitness = better result
• α and β control the tradeoff b/w accuracy and simplicity.

Code:

pip install numpy pandas scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# --------------------------
# Grey Wolf Optimizer (GWO)
# --------------------------
def fitness_function(features, X_train, X_test, y_train, y_test):
    # Select only chosen features
    cols = [i for i in range(len(features)) if features[i] > 0.5]
    if len(cols) == 0:
        return 0
    X_train_sel = X_train[:, cols]
    X_test_sel = X_test[:, cols]

    # Train simple KNN classifier
    clf = KNeighborsClassifier(n_neighbors=3)
    clf.fit(X_train_sel, y_train)
    y_pred = clf.predict(X_test_sel)
    return accuracy_score(y_test, y_pred)

def GWO(X_train, X_test, y_train, y_test, n_wolves=10, max_iter=20):
    dim = X_train.shape[1]
    alpha_pos = np.zeros(dim)
    beta_pos = np.zeros(dim)
    delta_pos = np.zeros(dim)
    alpha_score = beta_score = delta_score = 0

    wolves = np.random.rand(n_wolves, dim)

    for t in range(max_iter):
        for i in range(n_wolves):
            fitness = fitness_function(wolves[i], X_train, X_test, y_train, y_test)
            if fitness > alpha_score:
                alpha_score, alpha_pos = fitness, wolves[i].copy()
            elif fitness > beta_score:
                beta_score, beta_pos = fitness, wolves[i].copy()
            elif fitness > delta_score:
                delta_score, delta_pos = fitness, wolves[i].copy()
```

```
        a = 2 - t * (2 / max_iter)  # Linearly decreasing

        for i in range(n_wolves):
            for j in range(dim):
                r1, r2 = np.random.rand(), np.random.rand()
                A1, C1 = 2 * a * r1 - a, 2 * r2
                D_alpha = abs(C1 * alpha_pos[j] - wolves[i][j])
                X1 = alpha_pos[j] - A1 * D_alpha

                r1, r2 = np.random.rand(), np.random.rand()
                A2, C2 = 2 * a * r1 - a, 2 * r2
                D_beta = abs(C2 * beta_pos[j] - wolves[i][j])
                X2 = beta_pos[j] - A2 * D_beta

                r1, r2 = np.random.rand(), np.random.rand()
                A3, C3 = 2 * a * r1 - a, 2 * r2
                D_delta = abs(C3 * delta_pos[j] - wolves[i][j])
                X3 = delta_pos[j] - A3 * D_delta

                wolves[i][j] = (X1 + X2 + X3) / 3

        wolves = np.clip(wolves, 0, 1)
        print(f"Iteration {t+1}/{max_iter}, Best Accuracy: {alpha_score:.4f}")

    return alpha_pos, alpha_score


# --------------------------
# Run GWO on Iris dataset
# --------------------------
iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

best_features, best_acc = GWO(X_train, X_test, y_train, y_test)
selected_features = [i for i, val in enumerate(best_features) if val > 0.5]

print("\nBest selected features:", selected_features)
print("Best Accuracy:", best_acc)
```

Program 7

Parallel Cellular Algorithms and Programs

Algorithm:

Date :- 30/10/2025 27

Parallel Cellular Algorithm :- Inspired by cellular Automata - a model where space is divided into a grid of cells and each cell evolves over discrete time steps based on :-

(i) its current state, and
(ii) the state of its neighbouring cells

So, PCA extends this idea for parallel computation - every cell (or processor) updates its states simultaneously (in parallel) based on local interactions.

Algorithm :-

Input :-
(i) Grid of initial cells states.
(ii) Update rule function (f)
(iii) No. of iterations (T).

Output :- Final grid configuration after T iterations

Step1 :- Initialize the grid cells with initial cell states.

Step2 :- For each time step t = 1 to T:
(a) In parallel, for every cell i:
→ Read the states of i and its neighbours
→ Compute the new state new_state[i] = f(neighbourhood(i)).
(b) Synchronize all updates (so that all cells use the same generation's data)

(c) Replace old grid G_t with new grid G_t'.

3) Return the final grid configuration.

Pseudocode :-

Parallel Cellular_Algorithm (G_1, f, T)

Input:
G_1 – Initial grid (array of cells)
f – transition fn (update rules)
T – no. of time steps

Begin
for $t = 1$ to T do.
in parallel for each cell i in G_t do
$N_i \leftarrow$ Neighbourhood (i, G_t)
new_state (i) $\leftarrow f(N_i)$
using rule f
end parallel for
Synchronize all updates
$G_t \leftarrow$ new_state
grid to next generation
end for
return G_t.

End.

Transition fn :- A rule fn that decides the new state of a cell based on the current state of its neighbourhood.

Formally: $S_i(t+1) = f(N_i(t))$

- $S_i(t)$ = State of cell i at time t
- $N_i(t)$ = neighbourhood of i at time t
- f = update / transition fn.

Key Parameters used in this Algorithm :-

Parameter	Meaning	Example
Grid	initial grid of cells	[0, 1, 1, 0, 1]
State	Possible values a cell can take	{0, 1}
N_i	Neighbourhood of cell i	$\{i-1, i, i+1\}$
f	Transition fn (update rule)	$f(N_i)$
T	No. of iterations / time steps	$T = 10$
Parallel update	All cells updated simultaneously	Yes
Synchronization	wait for all cells before next step	Barrier step
Output	Final evolved grid	Updated cs.

MG.

Application :- Forest fire Simulation
(using Parallel Cellular Algorithm)

Input :-

- Grid (2D Array) :- Each cell contains one of three states — 'A' (Alive), 'B' (Burning), or 'D' (Dead)

Output :-

Step 1 :-

A A A A
A B A A
A A A A
A A A A

Step 2 :-

A B A A
B D B A
A B A A
A A B A

Step 3 :-

B D B A
D D D B
B D B A
B A D B.

MG
30/10/25.

Code:

```
import numpy as np
from multiprocessing import Pool

# Cell states
ALIVE = 'A'
BURNING = 'B'
DEAD = 'D'

def update_cell(args):
    i, j, grid = args
    if grid[i, j] == BURNING:
        return DEAD
    elif grid[i, j] == ALIVE:
        # Catch fire if at least one neighbor is burning
        neighbors = [(i-1,j), (i+1,j), (i,j-1), (i,j+1)]
        for ni, nj in neighbors:
            if 0 <= ni < grid.shape[0] and 0 <= nj < grid.shape[1]:
                if grid[ni, nj] == BURNING:
                    return BURNING
        return ALIVE
    else:
        return DEAD

def parallel_forest_fire(grid, steps):
    for step in range(steps):
        print(f"Step {step+1}:")
        for row in grid:
            print(' '.join(row))
        print()
        # Flattened cell indices for parallel processing
        cell_args = [(i, j, grid) for i in range(grid.shape[0]) for j in range(grid.shape[1])]
        with Pool() as pool:
            new_states = pool.map(update_cell, cell_args)
        grid = np.array(new_states).reshape(grid.shape)
    return grid

# Initial grid setup (4x4 example)
grid = np.array([
    [ALIVE, ALIVE, ALIVE, ALIVE],
    [ALIVE, BURNING, ALIVE, ALIVE],
    [ALIVE, ALIVE, ALIVE, ALIVE],
    [ALIVE, ALIVE, ALIVE, ALIVE]
])

# Run simulation for 5 steps
final_grid = parallel_forest_fire(grid, 5)
```

```
print("Final Grid after 5 steps:")
for row in final_grid:
    print(' '.join(row))
```